
WRO 2026
Future Engineers
Győr, Hungary
2026.06.26 – 2026.06.27

BITS

Table of contents

Hardware Documentation	1
1. Our Robot	1
1.1 Drive system	2
1.2 Power and Electronics	5
1.3 Sensors	10
1.4 Processing Units	12
1.5 Wheels.....	14
2. 3D Print.....	15
3. Cost of the Robot.....	17
4. Circuit Design.....	18
5. Assembly Guide	19
6. Pictures:.....	20
Software Documentation	27
7. System Architecture	27
7.1 Overview.....	27
7.2 Blackboard Pattern	27
7.3 Event System	28
7.4 Thread Model	28
7.5 Data Flow.....	29
8. Coordinate Systems	30
World Frame.....	30
Body Frame.....	30
Planning Grid	30
9. Components	31
9.1 Lidar	31
9.2 Camera	31
9.3 Arduino	32
9.4 Ticker	33
10. Processors.....	34
10.1 LidarProcessor	34
10.2 CameraProcessor.....	35

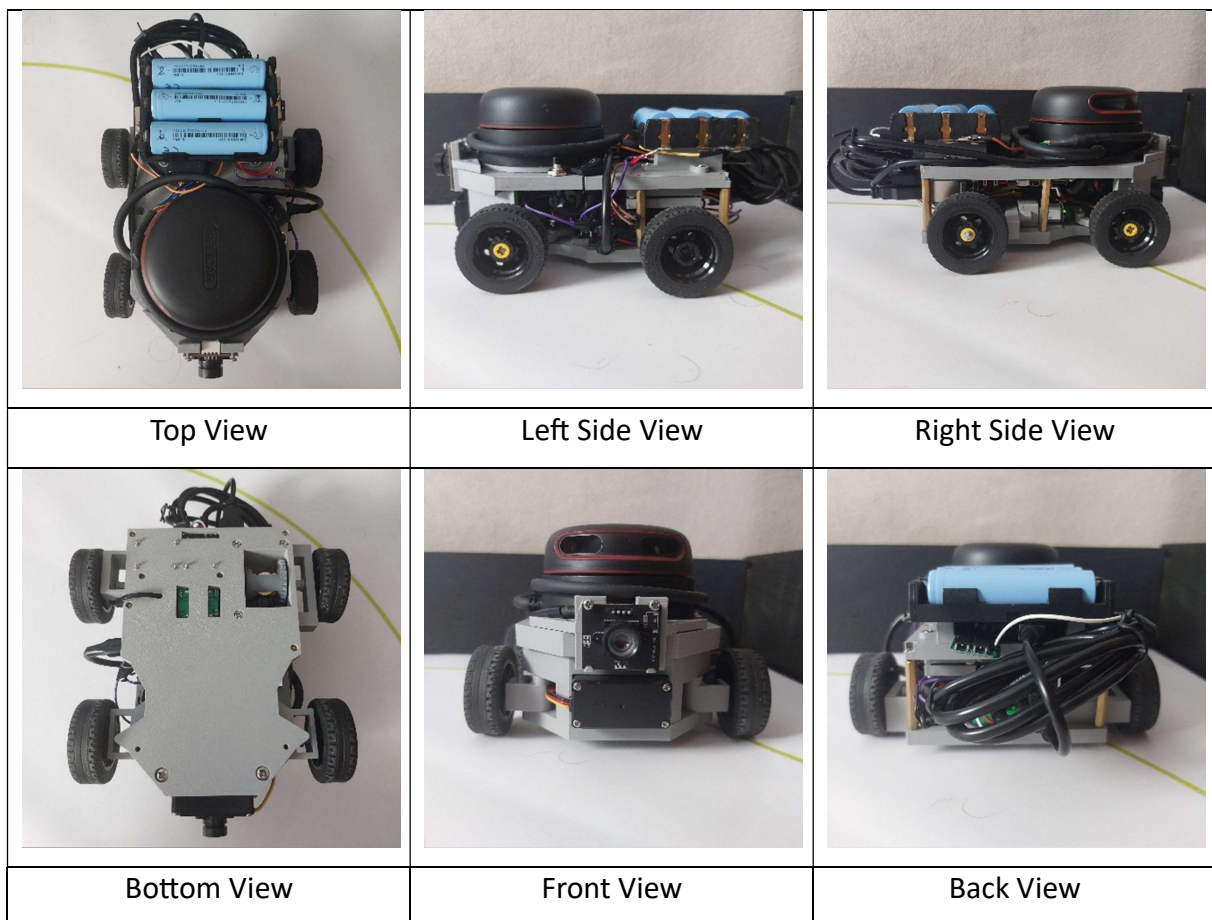
10.3 SemanticClassifier	36
10.4 ArduinoProcessor	37
10.5 CollisionGuard	37
10.6 DirectionDetector	38
10.7 PathPlanningProcessor	38
11. Control Layer	41
11.1 FieldMap.....	41
11.2 EgoInformation.....	42
12. Arduino Firmware.....	44
12.1 Protocol	44
12.2 Navigator	45
12.3 Motor.....	46
12.4 Steering.....	47
12.5 Encoder.....	47
12.6 IMU.....	48
13. Calibration	49
13.1 Camera Intrinsics	49
13.2 Camera Extrinsics	50
13.3 Obstacle Colour Ranges.....	50
13.4 Odometry Noise	51
13.5 IMU Noise.....	52
13.6 LiDAR Noise	52

Github: <https://github.com/ErdelyiAndras/wro-future-engineers-2026>

Open Challenge: <https://youtu.be/fN8b082LGhQ>

Hardware Documentation

1. Our Robot



1.1 Drive system

DC motor: N20E-12-500

Specs:

Voltage: 12 V	
Speed: 500 RPM	
Torque: 41 mNm	
Current: 100 mA	
Dimensions: 26×10×12 mm	
Shaft: 3 mm diameter, D-spring contact interface	

Due to its compact size and adequate torque, the N20E motor was selected to drive the robot. It features a built-in encoder to provide precise feedback. The motor connects via a 3D-printed axle to the LEGO gears and subsequently to the LEGO differential. The application of differential gearing is crucial, as it minimizes wheel slippage during cornering. The motor is mounted to the differential housing using a LEGO style bracket, and the housing itself is secured to the baseplate with screws.

Previous tries:

We tried to use another version of this motor, the N20E-12-1000, but it couldn't provide us sufficient torque to be able to move the robot consistently.

Mounting:

- 3 pcs of M3*3*4,2 mm inserts
- 3 pcs of M3*8 mm screws

Servo: HS-485HB

Specs:

Voltage: 4,8 - 6V	
Speed (60°): 0,22 – 0,18s	
Torque range: 4,8 – 6 Kgcm	
Current draw (idle): 10 mA	
Stall current draw: 1200 mA	
Dimensions: 39.8 x 19.8 x 38.0 mm	
Weight: 45 g	

The HS-485HB is a small servo motor that has enough torque to move our steering mechanism and is small enough to fit on the front of the robot.

Our robot utilizes an Ackermann steering geometry. Ackermann steering geometry is a geometric arrangement of linkages in the steering of a car or other vehicle designed to solve the problem of wheels needing to trace circles of different radii when turning.

The Problem it Solves

When a vehicle turns a corner, the inside wheels travel along a tighter path than the outside wheels. Because the inside wheel is closer to the center of the turn, it must turn at a sharper angle than the outside wheel. If both wheels turned at the exact same angle, the tires would scrub, slide sideways, and cause unnecessary wear and tear.

How it Works

Ackermann steering achieves the correct angles by angling the steering arms inward.

The Alignment: If you draw a line from the steering knuckles (the pivot points of the wheels) through the tie-rod ends, those lines will intersect exactly at the center of the rear axle when the wheels are pointing straight ahead.

The Dynamic: When you turn the steering wheel, this angled linkage causes the inside wheel to turn more sharply than the outside wheel.

The Result

By ensuring that all four wheels trace paths around a single, shared center point, the vehicle experiences:

Smoother turning with minimal tire slippage or scrubbing.

Predictable handling and better directional stability.

Mounting:

- Servo mount:
 - 2 pcs M4*5*5mm inserts
 - 2 pcs M4*8 mm screws
- Servo motor:
 - 2 pcs M4*6*5mm inserts
 - 2 pcs M4*6 mm screws

1.2 Power and Electronics

Power source: EVE INR18650 + BMS 3S 20A

Battery specs:

Capacity: 3500 mAh	
Current: 10 A	
Voltage: 4,2 V	
Nominal voltage: 3,7 V	
BMS specs:	
Protection: Overcurrent, Over-discharge, Overcharge, Short circuit	
Operating Current: 10A	
Dimensions: 59 x 20 x 3.7mm	
Max. Pulse Current: 20A	

The robot's power supply is provided by a custom-built, series-connected (3S1P) battery pack consisting of three 18650 cells. A BMS (Battery Management System) is integrated into the system to ensure stable output voltage and current, while protecting the cells and the circuit against overcurrent, over-discharge, overcharge, and short circuits. The Raspberry Pi 5 requires 5V and 5A to operate, its power supply is managed by an XL4015-STDN-PSU DC-DC step-down (buck) converter. The camera, LiDAR, and Arduino receive their power directly from the Raspberry Pi via USB ports. The servomotor is powered by a separate DSN-1504 converter, with its output set to approximately 5V. The other electronic components receive power from the Arduino's pins.

Mounting:

- 4 pcs M3*5*4,2 mm inserts
- 4 pcs M3*8 screws
- 4 pcs M3*4*4,2 mm inserts
- 4 pcs M3*6 screws

Stepdowns:

MINI560-5V-5A

Specs:

Input voltage: 4 – 36 V	
Output voltage: 5 V	
Current: 5 A	
Dimensions: 29 x 18 x 5,4 mm	

This stepdown is used for the Raspberry Pi 5 to supply 5 V and 5 A for it.

Previous tries:

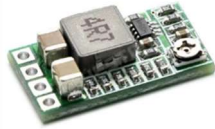
The XL4015-STDN-PSU DC-DC converter was unable to supply sufficient current to meet the demands of the Raspberry Pi. This power deficit resulted in voltage drops, triggering unstable behavior and intermittent, random system shutdowns.

Mounting:

- 2 pcs M3*5*4,2mm inserts
- 2 pcs M3*8 mm screws
- 2 pcs M2,5*4*3,5 mm inserts
- 2 pcs M2,5*10 mm screws

DSN-1504-3A

Specs:

Input voltage: 4,5 – 20 V	
Output voltage: 0,8 – 18 V	
Current: 3 A	
Dimensions: 22 x 17 x 4 mm	

This stepdown is used for the servo motor to supply sufficient power for it.

Mounting:

- 2 pcs M3*3*4,2 mm inserts
- 2 pcs M3*6 mm screws

Motor Controller:

DRV8871-MK:

Voltage: 6,5 – 45 V	
Max. Current: 3,6 A	
Dimensions: 25 x 21 x 9.5 mm	

The DRV8871-MK is a DC motor driver primarily used to control the speed, direction, and braking of the DC motor.

Previous ideas:

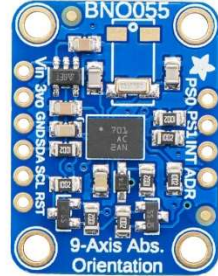
We tried the L298N motor driver, but it was proven to be overkill for our project. That controller can manage 2 DC motors at the same and it's also huge in size compared to our robot.

Mounting:

- 2 pcs M2*10 mm screws
- 2 pcs M2 nuts

IMU:

BNO055:

Type: 9 axis IMU	
Voltage: 3,3 V	
Communication Interface: I ² C	

The BNO055 provides real-time positional data, which helps the robot maintaining correct direction and smooth out steering adjustments.

Mounting:

- 4 pcs M2*10 mm screw
- 4 pcs M2 nut

1.3 Sensors

RPLIDAR A2M8

Specs:

Distance Range: 0.15 – 12 m	
Sampling Frequency: 8 kHz	
Scan Rate: Adjustable from 5 to 15 Hz	
Angular Range: 360 degrees	
Angular Resolution: 0.9°	
Dimensions: 76 mm x 76 mm x 41 mm	
Weight: 190 grams (Sensor core)	

This LIDAR allows us to make precise measurements of obstacles around the robot. The high sampling frequency allows real-time mapping and obstacle avoidance.

Previous tries:

The RPLIDAR A3M1 LiDAR sensor did not provide sufficient point cloud density; consequently, precise navigation would not have been feasible due to its lack of detailed data.

Mounting:

- 4 pcs M3*8mm screws

Camera:**DFRobot FIT0701****Specs:**

Voltage: 5V	
Resolution: 640 x 480 px	
Dimensions: 30 x 25 x 21,4 mm	

The camera is used for obstacle and color detection.

Mounting:

- 2 pcs M3*5*4,2 inserts
- 2 pcs M3*8 mm screws
- 2 pcs M2,5*5*3,5 mm insertss
- 2 pcs M2,5*8 mm screws

1.4 Processing Units

Raspberry Pi 5 8 GB

Specs:

Voltage: 5 V	
RAM: 8 GB	
Processor: ARM Cortex-A76	
Ports (we use): 2x USB 3.0, 1x USB 2.0	
Dimensions: 85 x 56 x 17 mm	

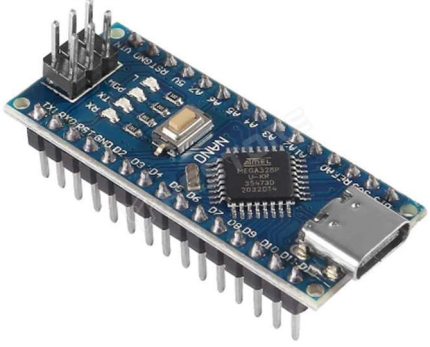
The Raspberry Pi 5 serves as our main computational unit. It processes real-time LIDAR and camera data and manages tasks like mapping and path planning.

Mounting:

- 4 pcs M2,5*8mm screws
- 4 pcs M2,5*5*3,5mm inserts

AR-NANOCH-UNS-TYPE-C

Specs:

Microcontroller: Atmel Atmega32	
Voltage: 5 V	
Dimensions: 45 x 18 mm	

The Arduino Nano serves as our secondary controller managing tasks such as motor control, servo steering and sensor inputs. It communicates with the Raspberry Pi 5 to ensure maximum control.

Mounting:

- 2 pcs M3*3*4,2 mm inserts
- 2 pcs M3*6 mm screws

Further ideas:

Level shifter (LS-BIDI-4) → USB:

Previously the Raspberry and the Arduino communicated via a level shifter inside the robot, however it required 10 cables in total to connect them. So to save space and make it easier to take apart the robot we switched to a USB cable.

1.5 Wheels

Initially, wheels with a larger diameter and wider profile were deployed. However, due to excessive friction they were replaced with smaller, narrower alternatives. This swap was further justified by the fact that the larger wheels exhibited a disproportionately high rotational deadzone relative to their size.

2. 3D Print

Printer Selection

The **Bambu Lab P1S** FDM printer was selected for this project to enable systematic material experimentation and validation across multiple types of filaments. The printer's fully enclosed thermal chamber provides precise temperature regulation, minimizing warping and ensuring dimensional consistency across production runs—critical requirements for functional robotics components.

Key Specifications

- Fully enclosed heated chamber with active thermal management
- Multi-material capability (PLA, PETG, ASA, ABS)
- High positional accuracy suitable for mechanical assemblies

Material Strategy

A three-stage material selection approach was employed:

Prototyping Phase

Material: PLA

- Used for rapid iteration and design validation
- Lower processing temperature reduces printer stress during concept testing

Production Phase

Two materials were selected for the final assembly based on functional requirements:

ABS (Acrylonitrile Butadiene Styrene)

Application: Thermal-critical components (e.g., Raspberry Pi 5 mounting plate)

- Superior heat resistance and thermal stability
- Upper service temperature: ~80–100 °C continuous operation
- ABS was selected to provide a substantial safety margin and ensure thermal resilience under worst-case scenarios without risk of material softening or creep.

PETG-HF (High-Flow PETG)

Application: Functional mechanical assemblies (differential gearbox housing, servo mount)

- Enhanced layer adhesion and impact resistance compared to standard PETG
- Adequate thermal resistance for non-critical zones
- Superior dimensional retention and reduced creep under cyclic loading
- Justification: PETG is highly durable and tends to bend slightly rather than shatter under stress. This makes it ideal for parts that endure drops, vibrations, or repeated wear. The layer adhesion is also exceptional parts are less prone to splitting along weak seams when under load.

Quality Assurance

All printed components underwent:

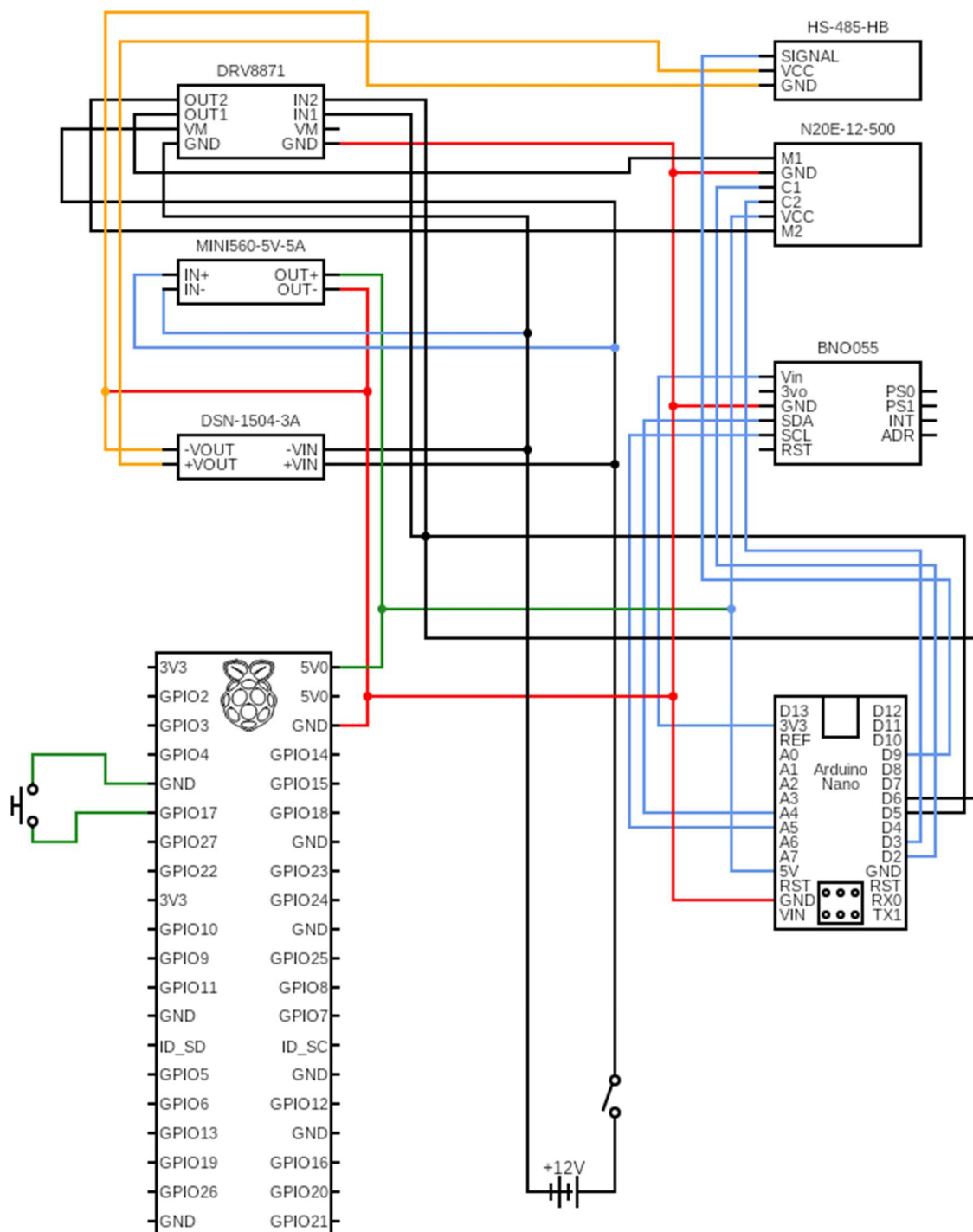
- Visual inspection for layer uniformity and surface finish
- Dimensional verification against CAD models (tolerance stack-up analysis)
- Mechanical fit testing of assemblies prior to integration

3. Cost of the Robot

Product name	Price (HUF)	Price (EUR)
Raspberry Pi 5 8GB	69.990,00	191,75
AR-NANOCH-UNS-TYPE-C	1.992,45	5,45
BNO055	1.7990,00	49,28
DRV8871-MK	1.758,91	4,81
DFRobot FIT0701	3.042,95	8,33
RPLIDAR A2M8	98.065,86	268,67
DSN-1504-3A	444,11	1,21
MINI560-5V-5A	1.457,76	3,99
INR EVE 18650	3.000,00	8,21
BMS 3S 20A	541,09	1,48
HS-485HB	8.918,16	24,43
N20E-12-500	4.370,7	11,97
LEGO Wheels	1.932,00	5,29
ST 1/BK	250,10	0,68
PBS-110-NO-BK	294,22	0,8
Total	214.048,31	586,43

1 EUR = 365 HUF

4. Circuit Design



The camera, the Arduino and the LIDAR are connected to the Raspberry via USB.

To make repairing and debugging easier we used color coded wires.

5. Assembly Guide

Print out each part with the right material:

Top and bottom plates: ABS

Every other part: PETG

For printing out the parts we used the base Bambu settings, since they're already decent enough.

Important Note:

Use a soldering iron to put the inserts into the 3D printed parts based on the pictures given.

Recommended temperatures:

- PLA: 130 °C
- PETG: 180 °C
- ABS: 200 °C

Legend:

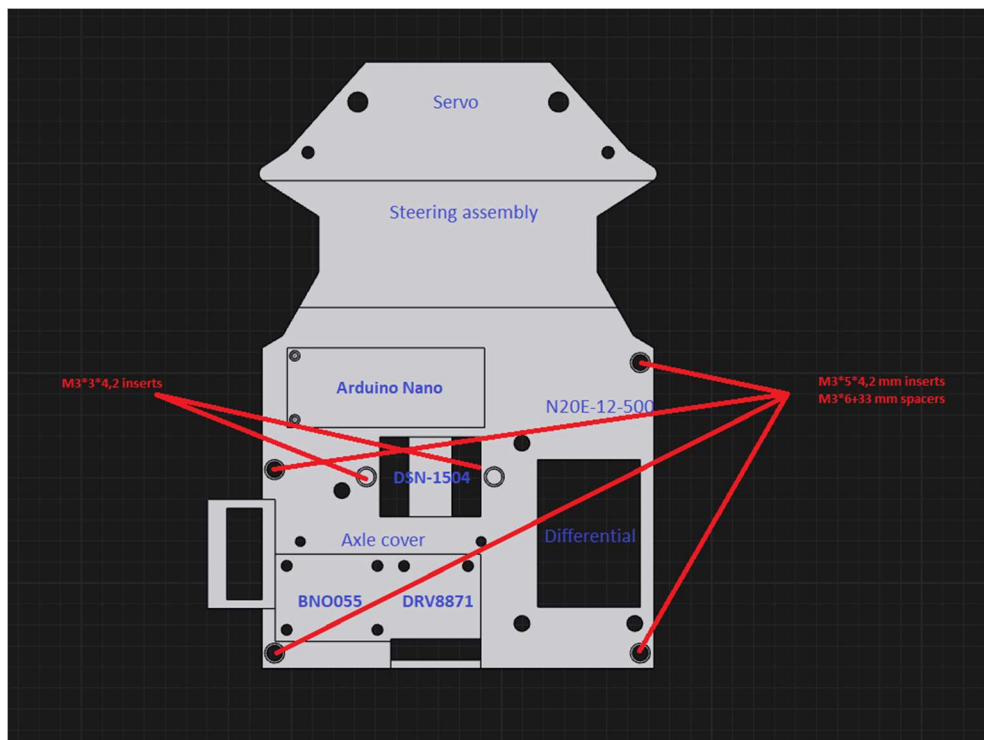
- Blue: Part name
- Red: Size of the inserts and screws

Instruction:

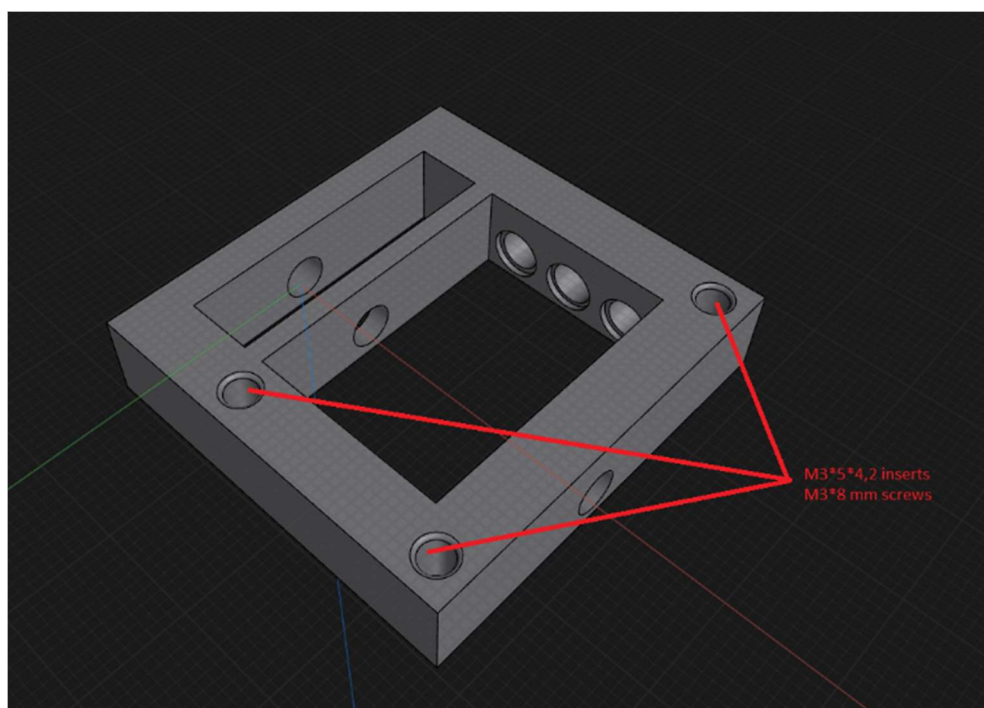
Place every part, insert and screw exactly where the instruction says.

6. Pictures:

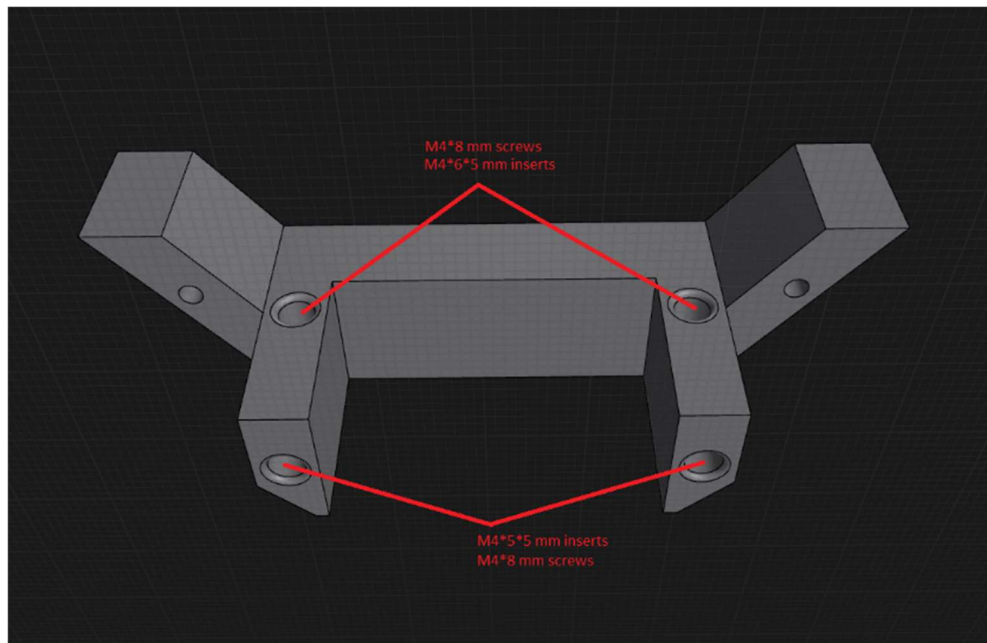
Bottom Plate:



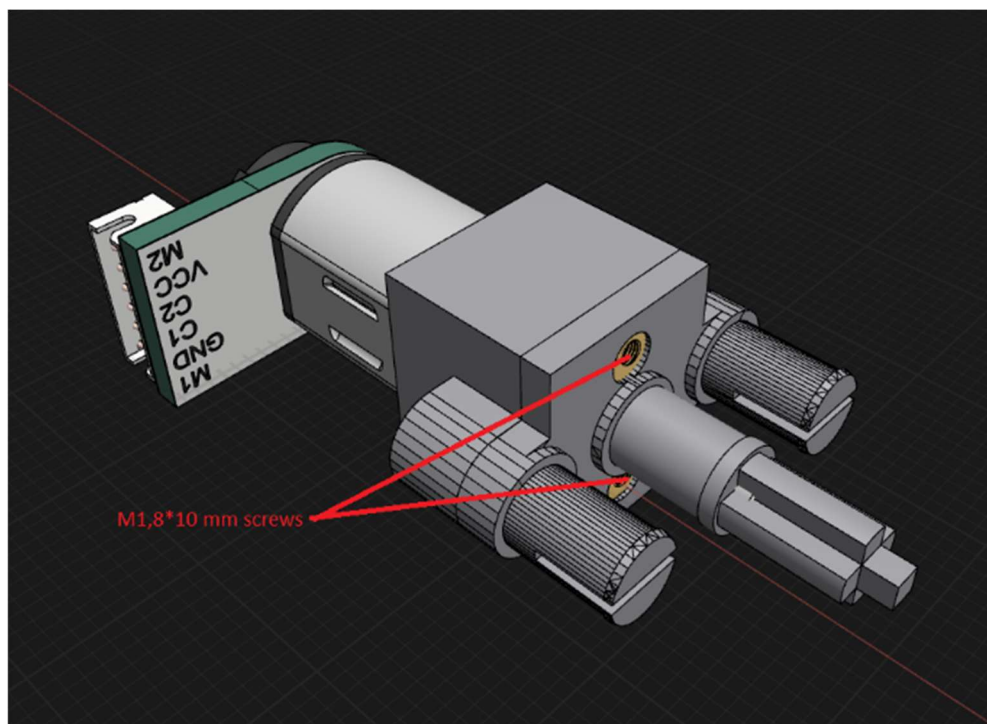
Differential Housing:



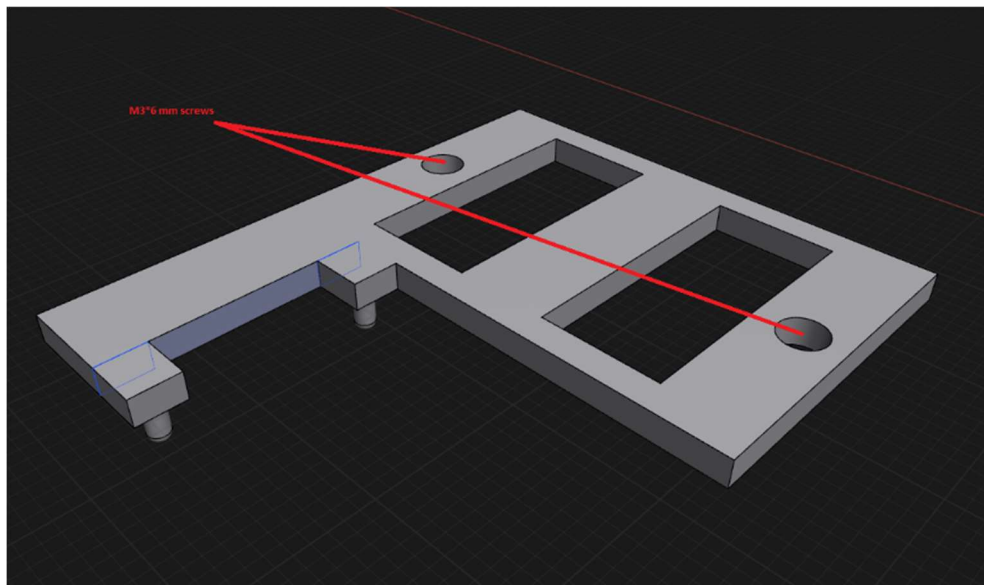
Servo Holder:



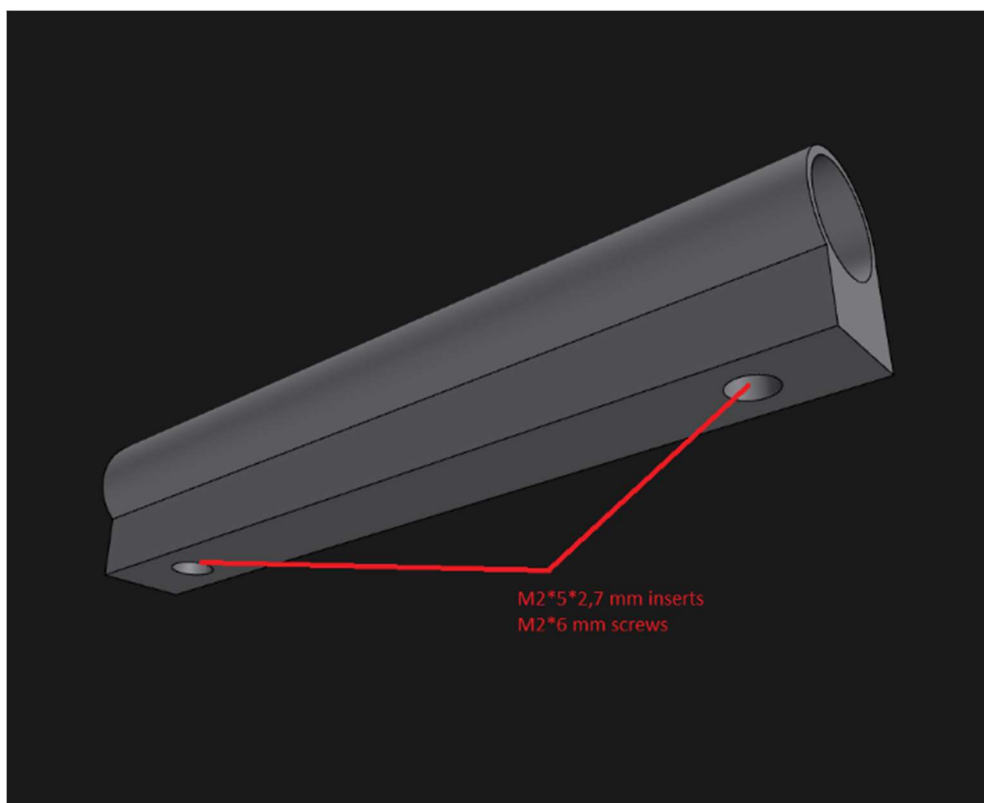
DC Assembly:



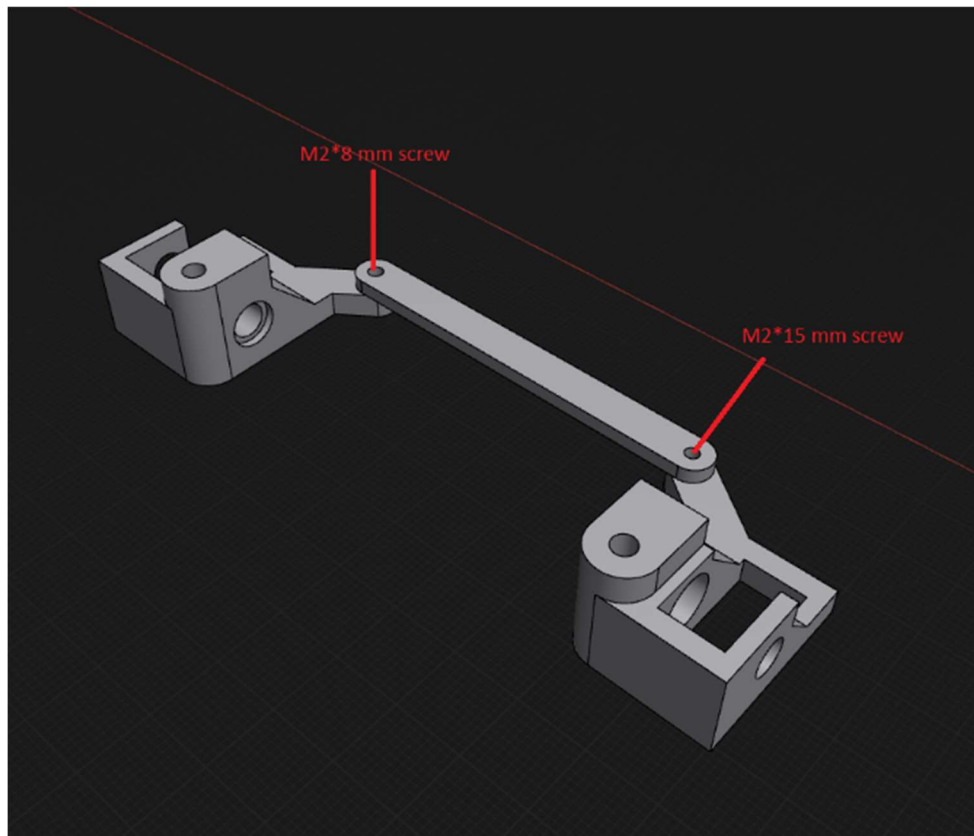
Arduino Holder:



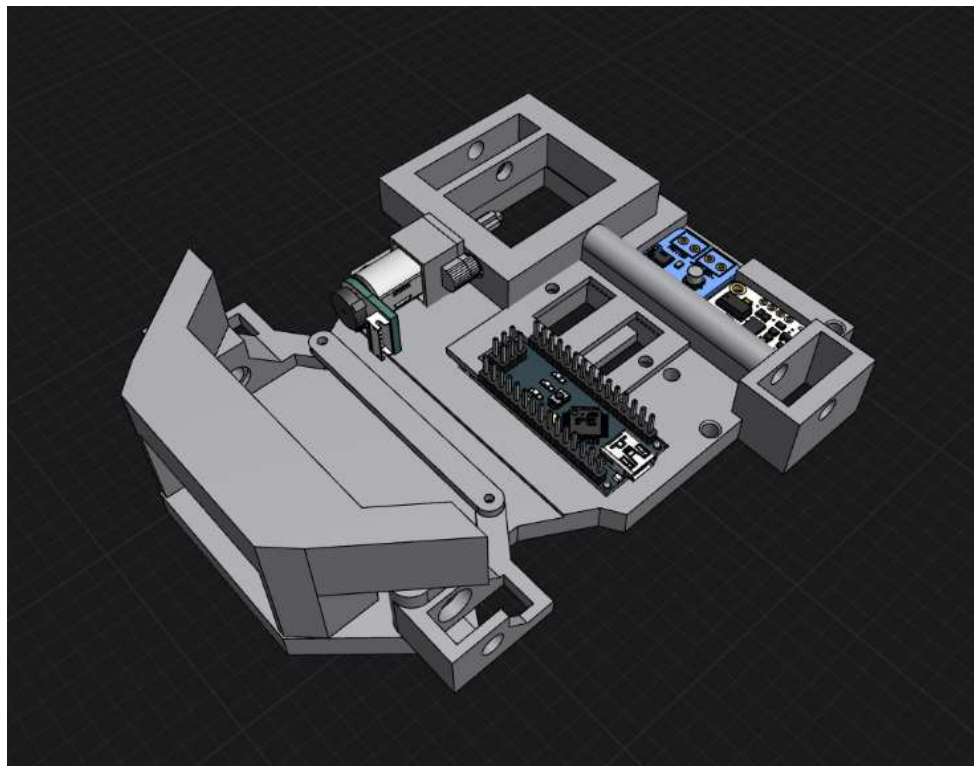
Axle Cover:



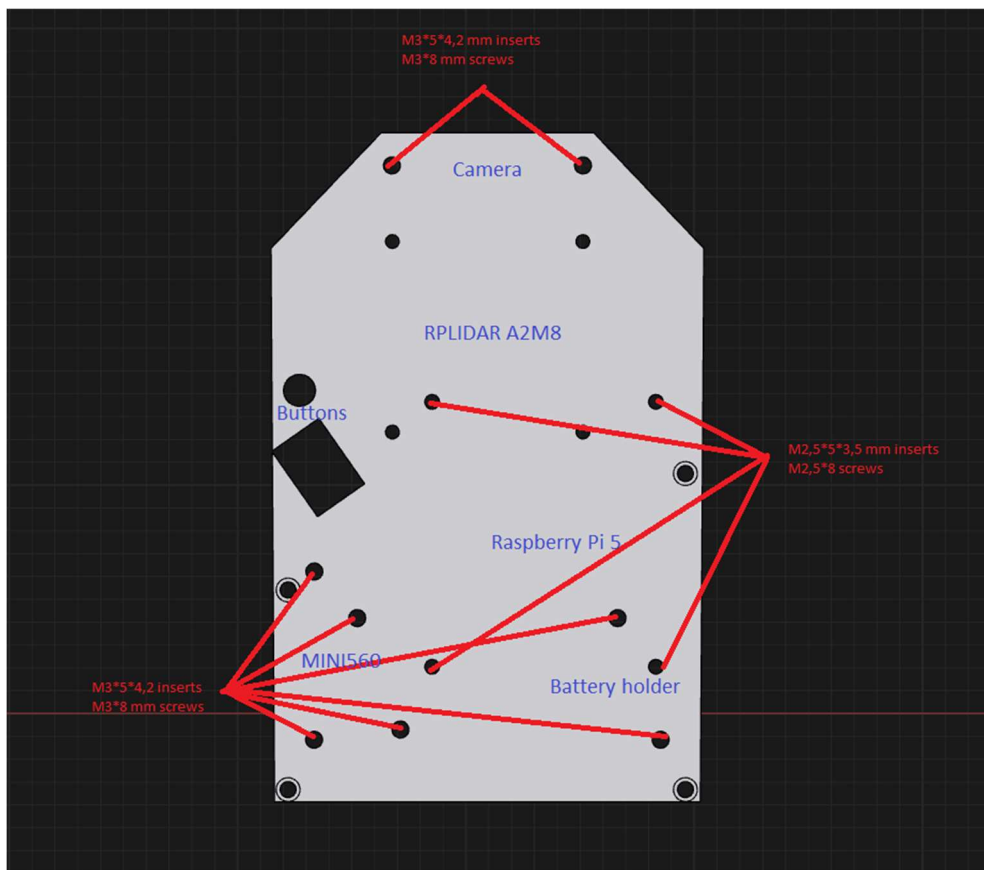
Steering Assembly:



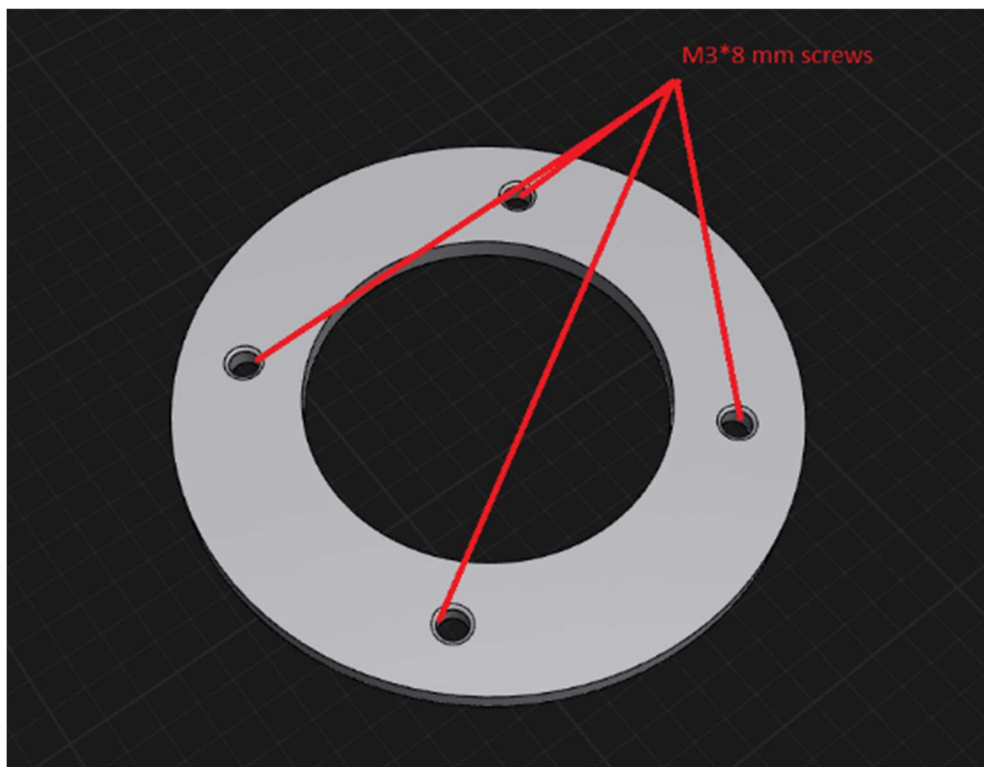
Fully Assembled Bottom Part:



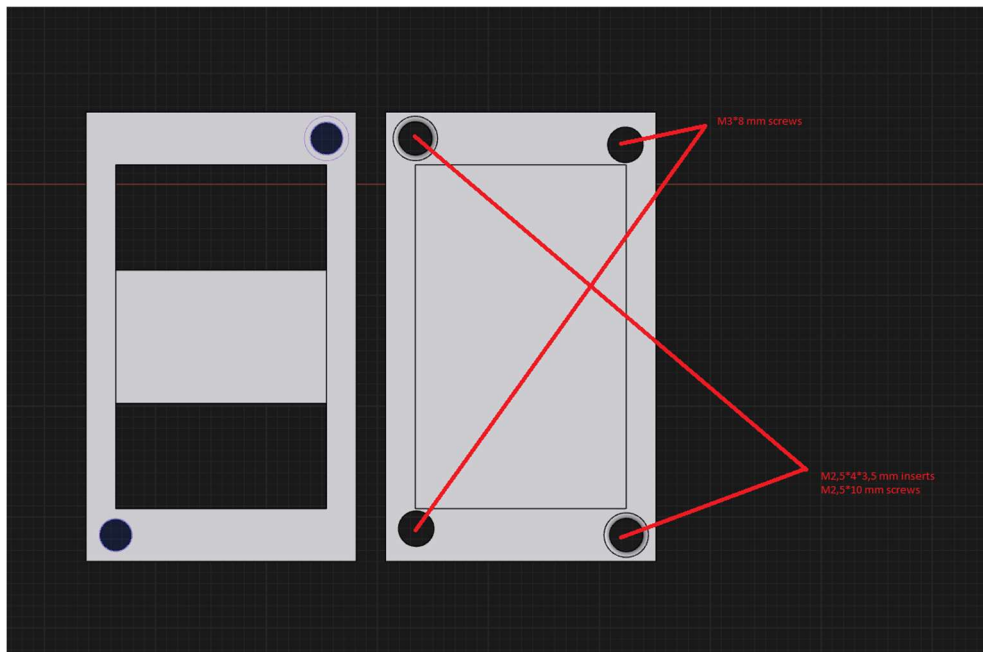
Top Plate:



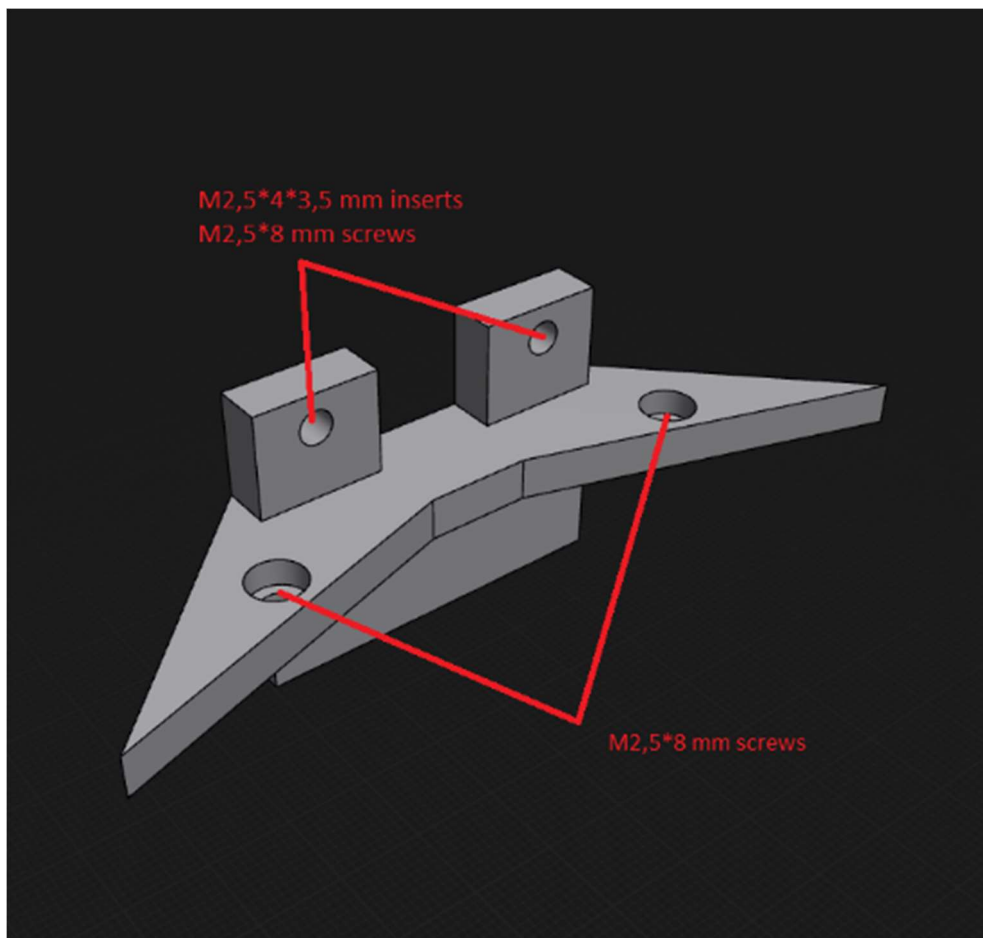
Lidar Holder:



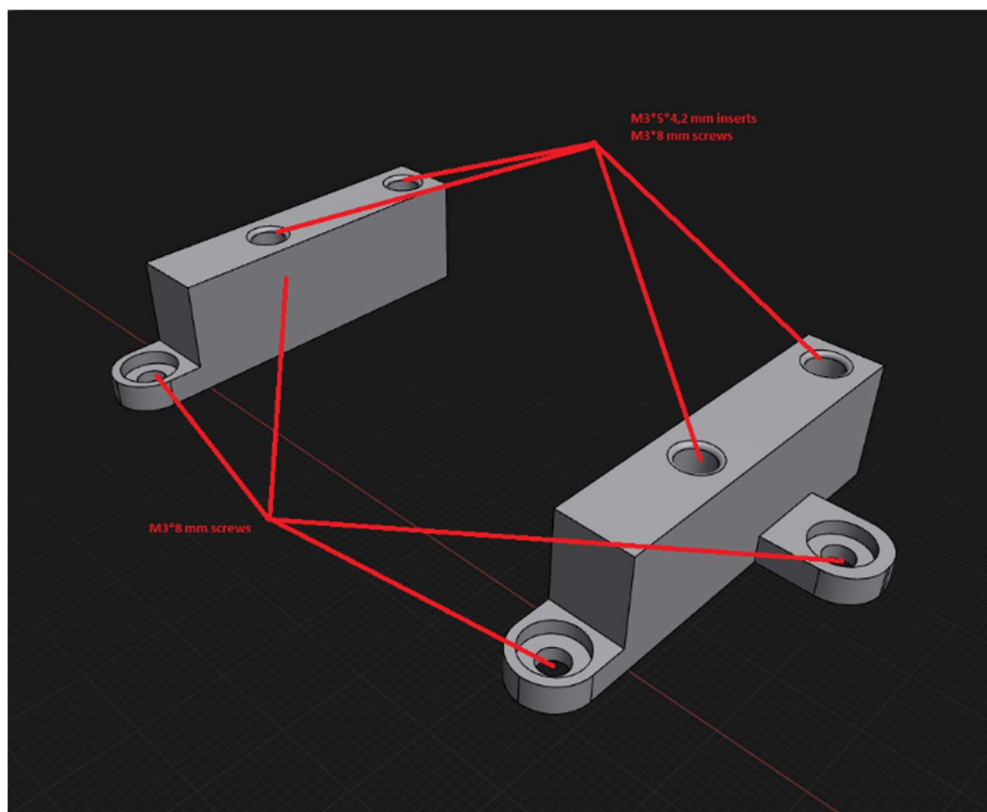
Stepdown Holder:



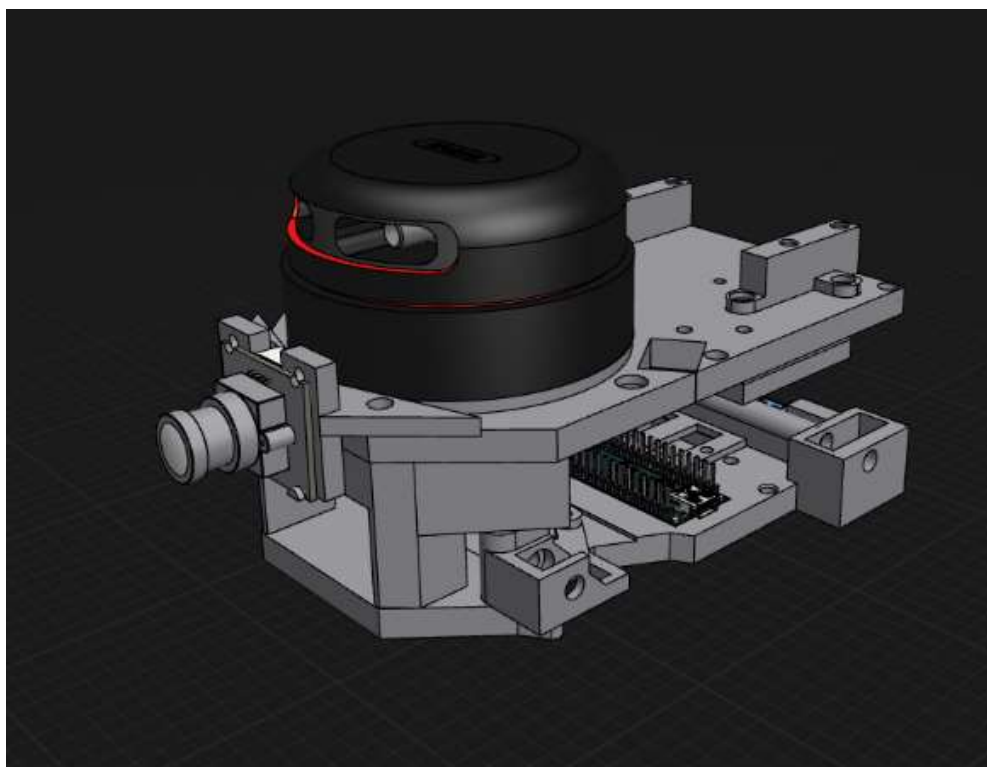
Cam Holder:



Battery Holder:



Assembled Robot:



Software Documentation

7. System Architecture

7.1 Overview

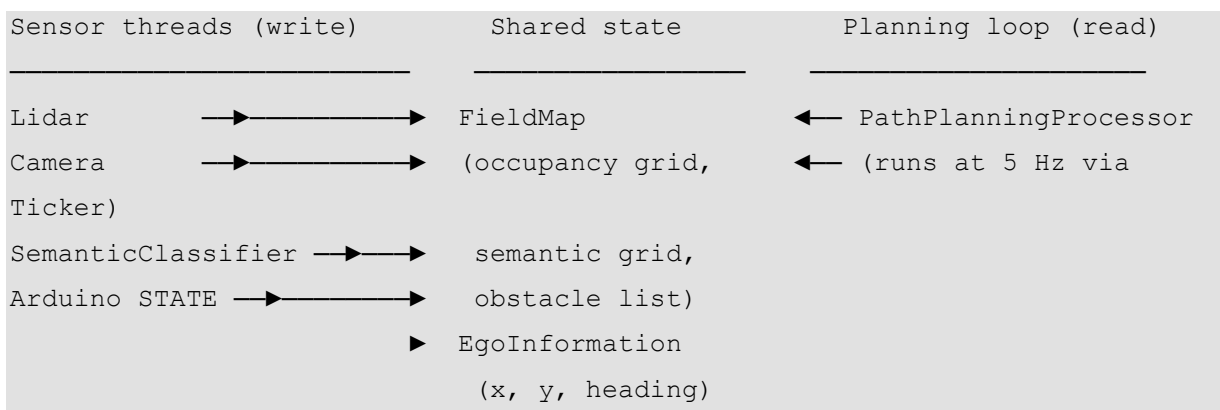
The software is split across two processors with a clean boundary defined by responsibility:

- **Raspberry Pi** — perception, localization, map building, and motion target generation. Runs Python with background threads.
- **Arduino Nano** — real-time motion execution. Owns the IMU (BNO055) and rear-axle encoder. Runs a control loop at ≥ 50 Hz.

This split exists because smooth motion control requires deterministic timing that a general-purpose OS cannot guarantee. The Arduino runs a tight loop reading sensors and commanding actuators, while the Raspberry Pi handles the heavier computation of mapping and planning at a lower, irregular rate.

7.2 Blackboard Pattern

The Raspberry Pi side uses a **blackboard pattern**: all sensor threads write to two shared data structures (`FieldMap` and `EgoInformation`), and the planning loop reads from them independently. Both classes are thread-safe: locks are held only for the duration of a copy — never during computation — so each thread runs at its own natural rate without blocking others.



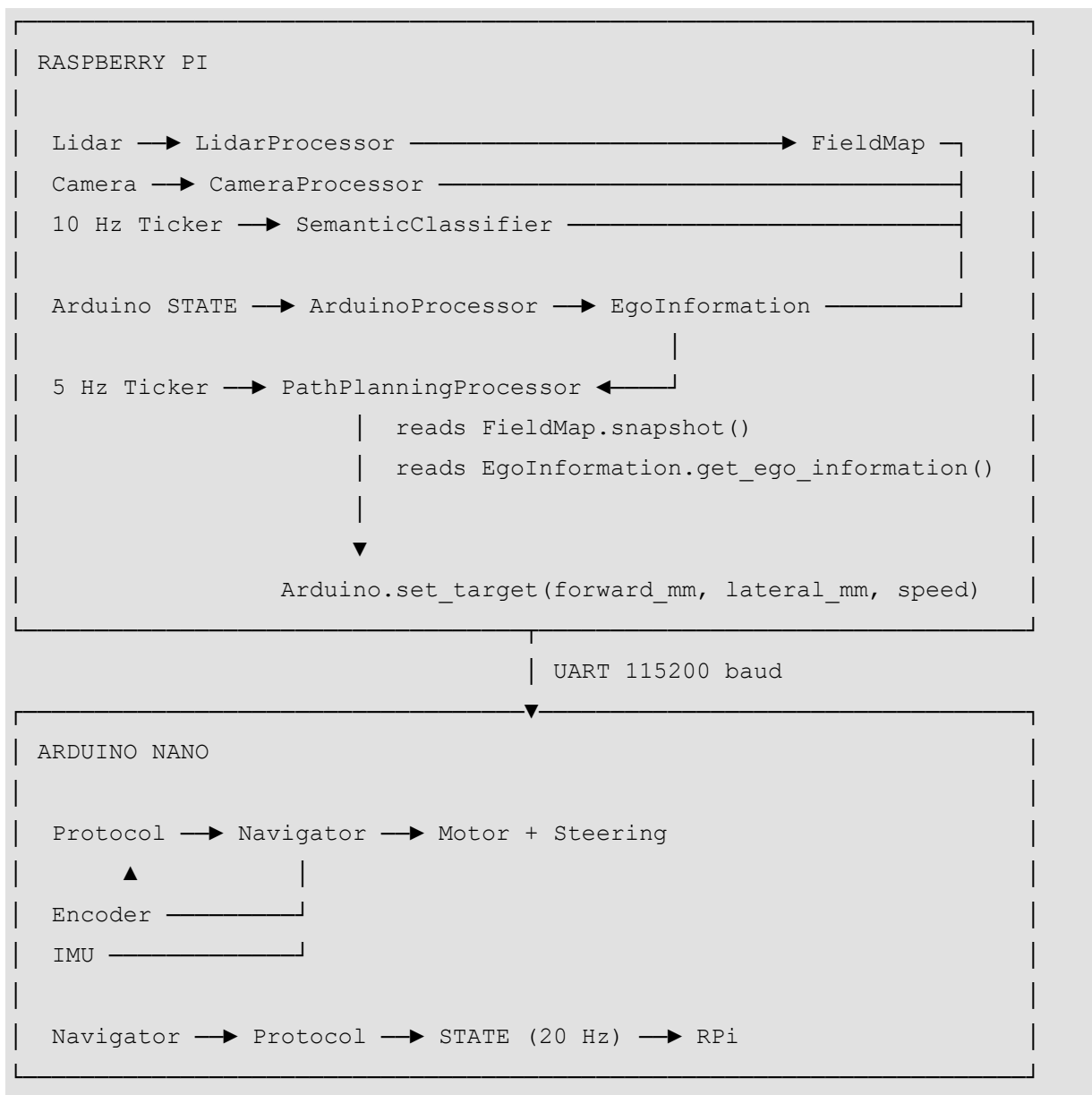
7.3 Event System

Sensors do not call processors directly. Each component fires an Event (a simple callable list) when new data arrives. Processors subscribe to events at startup. This decouples the producer from the consumer and allows multiple processors to subscribe to the same event — for example, both `LidarProcessor` and `CollisionGuard` subscribe to the lidar's `on_scan` event. The event system also enabled to wire in real-time visualisation tools by subscribing to the adequate event.

7.4 Thread Model

Thread	Rate	Source	Writes to
Lidar background	~7 Hz (full rotations)	<code>Lidar.on_scan</code>	FieldMap occupancy, EgoInformation
Camera background	30 fps	<code>Camera.on_frame</code>	FieldMap obstacle color votes
Classification ticker	10 Hz	<code>Ticker.on_tick</code>	FieldMap semantic labels
Planning ticker	5 Hz	<code>Ticker.on_tick</code>	<code>Arduino.set_target()</code>
Arduino serial reader	continuous	<code>Arduino.on_state</code>	EgoInformation odometry

7.5 Data Flow



8. Coordinate Systems

World Frame

Used by the Raspberry Pi for all map and planning operations.

- **Origin:** field centre. In the `FieldMap` grid this corresponds to (3000, 3000) mm, placing the origin at the grid centre.
- **+X axis:** east (right in top-down view).
- **+Y axis:** north (up in top-down view).
- **Heading:** radians, 0 = +X axis, positive counter-clockwise.

Body Frame

Used by the Arduino for motion commands.

- **Origin:** centre of the rear axle.
- **+X axis:** forward (robot's direction of travel).
- **+Y axis:** left.
- `SET_TARGET` carries `forward_mm` and `lateral_mm` in this frame.

Planning Grid

Used internally by `PathPlanningProcessor`.

- `FieldMap` fine grid: 1200×1200 cells at 5 mm/cell, covering a 6000×6000 mm area.
- Planning grid: downsampled by factor $K = 5$, giving 240×240 cells at 25 mm/cell.

9. Components

Components are hardware drivers. Each runs in a background daemon thread and fires events when new data is ready. All components implement a common lifecycle: `__enter__` opens the hardware connection, `start()` launches the background thread, `stop()` joins the thread, and `__exit__` closes the connection. A failed setup triggers teardown automatically, so resources are never leaked.

9.1 Lidar

File: `src/rpi/components/Lidar.py`

Drives the RPLIDAR sensor over serial. On each complete 360° rotation, fires the `on_scan` event with a NumPy array of shape `(N, 3)` containing `(angle_rad, distance_mm, quality)` for every valid return in that scan.

Invalid returns (zero quality or zero distance) are filtered before the event fires. The motor PWM is set to 660 out of 1023 to run the spinner at a rate suitable for the A2M8 sensor. If a serial exception occurs the thread flushes the buffer and retries after a short pause, so temporary USB disconnections are recovered automatically.

Parameter	Value
Port	<code>/dev/lidar</code> (symlinked)
Baud rate	115200
Motor PWM	660
Scan mode	Normal
Event	<code>on_scan(scan: np.ndarray)</code>

9.2 Camera

File: `src/rpi/components/Camera.py`

Captures frames from a USB camera using OpenCV `VideoCapture`. Each frame is rotated 180° (the camera is mounted upside down) and passed to the `on_frame` event as a BGR NumPy array. The thread loops continuously as fast as the camera delivers frames.

Parameter	Value
Port	Device index 0
Resolution	640×480
Target FPS	30
Event	<code>on_frame(frame: np.ndarray)</code>

9.3 Arduino

File: `src/rpi/components/Arduino.py`

Manages UART communication with the Arduino Nano. The background thread reads incoming bytes continuously, parses the framed protocol (described in section 6.6), and fires `on_state` on each valid STATE message.

Two send methods are available to the rest of the system:

- `set_target(forward_mm, lateral_mm, speed)` — sends a SET_TARGET frame telling the Arduino to pursue a body-frame offset at the given speed. Thread-safe; protected by a write lock.
- `stop()` — sends a STOP frame causing an immediate unconditional halt on the Arduino.

Parameter	Value
Port	<code>/dev/arduino</code> (symlinked)
Baud rate	115200
Event	<code>on_state(heading, speed, steering, distance)</code>

9.4 Ticker

File: `src/rpi/components/Ticker.py`

A wall-clock timer that fires its `on_tick` event at a fixed interval. Used to drive the `SemanticClassifier` at 10 Hz and the `PathPlanningProcessor` at 5 Hz, independent of any sensor arrival rate. Uses `threading.Event.wait` for the sleep so it wakes up immediately on `stop()` rather than waiting out the full interval.

Parameter	Value
Classification interval	0.1 s (10 Hz)
Planning interval	0.2 s (5 Hz)
Event	<code>on_tick()</code>

10. Processors

Processors subscribe to component events and transform raw sensor data into higher-level state stored in `FieldMap` or `EgoInformation`. Each processor inherits from `Processor`, which calls a `_process()` method and catches exceptions so a crash in one processor does not kill the thread that called the event.

10.1 LidarProcessor

File: `src/rpi/processors/LidarProcessor.py`

Subscribes to: `Lidar.on_scan`

Converts each raw lidar scan into world-frame hit points and updates the occupancy grid.

Processing steps:

1. Read the current robot pose (`x`, `y`, `heading`) from `EgoInformation`. If the pose is not yet initialised, skip this scan.
2. Apply the mounting angle offset (-90°) to correct for the lidar's physical rotation relative to the robot's forward axis.
3. Apply the mounting position offset (`0`, `73.37 mm`) — the lidar centre is 73 mm ahead of the rear axle.
4. Rotate the corrected hit points from body frame into world frame using the current heading.
5. Call `FieldMap.update_occupancy()` with the robot's world position and the world-frame hit points. The occupancy update uses a log-odds model: hit cells receive `+0.85`, free cells along each ray receive `-0.40`, and all values are clamped to `[-2.0, 3.5]`.

Parameter	Value
Mounting angle offset	-90°
Mounting position offset	(0 mm, 73.37 mm) from rear axle

10.2 CameraProcessor

File: `src/rpi/processors/CameraProcessor.py`

Subscribes to: `Camera.on_frame`

Associates observed pixel colours with known obstacle locations and votes their colour into `FieldMap`.

Processing steps:

1. Read the current robot pose from `EgoInformation`.
2. Retrieve the current obstacle list from `FieldMap`.
3. For each obstacle, project its world-frame centroid into the camera image using the calibrated intrinsics and extrinsics. Skip obstacles that project behind the camera ($z \leq 0$) or outside the image bounds.
4. Sample a 20-pixel-radius patch around the projected point in HSV colour space.
5. Count the fraction of pixels matching the red and green HSV ranges. If either fraction exceeds the minimum ratio (20%) and is larger than the other, cast a vote for that colour via `FieldMap.vote_obstacle_color()`.

An obstacle's colour is confirmed only after accumulating 7 net votes in the same direction. Red votes decay green votes and vice versa, making the decision robust to frames where lighting conditions are ambiguous.

Parameter	Value
Sample radius	20 px
Minimum colour ratio	20 %
Votes required for confirmation	7 net

10.3 SemanticClassifier

File: `src/rpi/processors/SemanticClassifier.py`

Subscribes to: `Ticker.on_tick` (10 Hz)

Labels occupied grid cells as `WALL`, `OBSTACLE`, or `PARKING_WALL` by fitting geometric primitives to the accumulated lidar point cloud.

Pipeline:

Step 1 — DBSCAN clustering. Groups all occupied world-frame points into spatial clusters with $\epsilon = 70$ mm. This separates the continuous wall returns from isolated obstacle returns.

Step 2 — Cluster classification. A cluster is an obstacle candidate if its bounding box extent is ≤ 200 mm and it contains ≥ 5 points. Larger clusters are treated as wall geometry. Obstacle candidates that fall within 100 mm of any wall cell are rejected — they are likely noise on the wall surface, not free-standing pillars.

Step 3 — RANSAC line fitting. On the wall points, iteratively find the best-fit line (50 iterations), collect inliers within 10 mm of the line, and remove them from the remaining set. Repeat up to 10 times (8 walls + 2 parking walls). Each inlier set is split at gaps > 150 mm into sub-segments and each sub-segment is tested for linearity (eigenvalue ratio ≥ 0.92) and flatness (RMS distance ≤ 8 mm).

Step 4 — Wall and parking wall classification. Sub-segments spanning ≥ 600 mm total are labelled `WALL`. Short segments (80–260 mm) that come in pairs with gap 150–600 mm and are nearly parallel (angle $< 10^\circ$) are labelled `PARKING_WALL`.

Step 5 — Label propagation. Occupied cells adjacent to labelled wall or parking wall cells inherit the same label. This ensures that thick wall returns — where the laser hit slightly different depths on different scans — are all consistently labelled.

10.4 ArduinoProcessor

File: `src/rpi/processors/ArduinoProcessor.py`

Subscribes to: `Arduino.on_state`

Bridges the Arduino's 20 Hz state stream into `EgoInformation`.

On each `STATE` message:

1. If the reported odometry distance `|distance| ≥ 0.5 mm`, call `EgoInformation.update_odometry(distance, steering)` to propagate the Kalman filter predict step using Ackermann kinematics.
2. Call `EgoInformation.update_imu(heading)` to apply the BNO055 absolute heading as a Kalman filter measurement correction.

10.5 CollisionGuard

File: `src/rpi/processors/CollisionGuard.py`

Subscribes to: `Lidar.on_scan`

A hard safety layer that operates directly on raw lidar data, independent of the map and the planner.

On every scan, it applies the mounting angle offset to convert raw angles to body-frame angles, then checks whether any return within the forward-facing cone ($\pm 30^\circ$ of the robot's forward axis) is closer than 150 mm. If so, it fires `on_collision`, which the `PathPlanningProcessor` receives to trigger a reverse-out maneuver.

Because this runs on every scan without waiting for a planning cycle, it reacts to obstacles that have not yet been incorporated into the occupancy grid.

Parameter	Value
Minimum forward clearance	150 mm
Forward cone half-angle	$\pm 30^\circ$
Event	<code>on_collision()</code>

10.6 DirectionDetector

File: `src/rpi/processors/DirectionDetector.py`

Subscribes to: `Lidar.on_scan`

Determines whether the robot is travelling clockwise or counter-clockwise around the track, and latches the result into `FieldMap` once confident.

As the robot drives the first segment, lidar hit points accumulate in the world frame (origin at start position). Points beyond ± 1500 mm on the X axis indicate which side of the field is filling in first. When one side has ≥ 80 points and leads the other by ≥ 40 , the direction is latched: more points at +X means clockwise, more at -X means counter-clockwise.

Once latched, the detector does nothing on subsequent scans. The direction is currently stored for map-building context; the path planner does not read it because its ray-cast goal selection finds the open corridor geometrically each cycle.

10.7 PathPlanningProcessor

File: `src/rpi/processors/PathPlanningProcessor.py`

Subscribes to: `Ticker.on_tick` (5 Hz)

The core navigation algorithm. On each tick it reads a consistent snapshot of the map and the current robot pose, then runs the full planning pipeline to produce a body-frame motion target sent to the Arduino.

Lap strategy:

- **Lap 1 (reactive):** The planner runs the A*-based algorithm described below and records the robot's path as a breadcrumb trail.
- **Laps 2–3 (route following):** After completing lap 1, the planner closes the recorded route into a loop and switches to pure-pursuit on the memorised path at higher speed. This is faster and more predictable than re-running A* every cycle on later laps.
- **Stop:** After 3 laps, the robot drives back to the start position and halts.

Reactive planning pipeline (lap 1):

Step 1 — Coarse grid. Downsample the 1200×1200 occupancy grid by factor $K = 5$, taking the max occupancy in each 5×5 block. Generate two coarse boolean masks: `blocked_all` (walls + obstacles + occupied cells) and `blocked_walls` (walls only, excluding obstacle cells). The distinction matters because obstacles should not generate a repulsive clearance gradient — the robot routes around them, not away.

Step 2 — Distance transforms. Compute the Euclidean distance transform of `blocked_all` to get collision clearance, and of `blocked_walls` to get corridor-centring clearance. Cache both; recompute only when the underlying blocked mask changes.

Step 3 — Breadcrumb trail. Stamp a decaying penalty in a rectangular patch behind and across-track of the robot's current position. This encodes "the direction I came from" and prevents the planner from selecting a backward goal or reversing into its own path.

Step 4 — Goal selection. Cast 17 rays spanning $\pm 80^\circ$ around the robot's heading. For each ray, step outward from a minimum distance to an adaptive maximum distance ($3.0 \times \text{corridor_clearance}$, clamped to 100–500 mm). Score each ray's furthest reachable traversable cell by $2.0 \times \text{reach} + 1.0 \times \text{corridor_clearance} - 1000 \times \text{max_breadcrumb}$. The ray reaching furthest into the open corridor wins. If no ray finds a free cell, the plan fails and the robot holds.

Step 5 — A*. 8-connected A* from the robot's current coarse cell to the goal cell. Edge cost:

```
cost(u → v) = move_cost          (1.0 cardinal, √2 diagonal)
               + clearance_penalty      (max(0, 100 - D[v]) /
100                                ×                                1.0)
               + backward_penalty      (150 × max(0, -dot(step,
heading)) /                                move_cost)
               + trail_penalty          (5.0 × trail[v])
```

The backward penalty discourages paths that double back, complementing the breadcrumb goal filter.

Step 6 — Path thinning. Reduce the A* path to the minimal set of waypoints where the path changes direction, using a greedy line-of-sight scan with Bresenham's algorithm on the inflated traversable mask.

Step 7 — Lookahead point. Walk 450 mm along the thinned path from the robot's position to find the pure-pursuit lookahead point.

Step 8 — Emit target. Convert the lookahead point to body frame and call `Arduino.set_target(forward_mm, lateral_mm, speed)`.

Parameter	Value
Planning rate	5 Hz
Downsample factor K	5 (25 mm/cell)
Robot radius (inflation)	150 mm
Forward cone half-angle	$\pm 80^\circ$
Number of rays	17
Lookahead L_min / L_max	100 / 500 mm
Clearance weight	2.0 reach + 1.0 clearance
Backward penalty	150.0
Trail decay per cycle	0.95
Pure-pursuit lookahead	450 mm
Reactive speed	80 mm/s
Route-following speed	100 mm/s

11. Control Layer

11.1 FieldMap

File: `src/rpi/control/FieldMap.py`

The shared spatial database. All sensor threads write to it; the planning loop reads from it. Three internal arrays are protected by a single lock:

- **Occupancy grid** (`np.float32`, 1200×1200) — log-odds probability of occupancy. Updated by `LidarProcessor` using a Bresenham ray-march to mark free cells along each ray and occupied at each hit. Values clamped to `[-2.0, 3.5]`.
- **Semantic grid** (`np.uint8`, 1200×1200) — cell labels: UNKNOWN (0), WALL (1), OBSTACLE (2), PARKING_WALL (3). Updated by `SemanticClassifier`.
- **Obstacle list** — tracked pillars, each with a set of occupied cells, red/green vote counts, and a latched pass-side decision (frozen direction once the colour is confirmed).

Key methods:

- `update_occupancy(ego_pos, hit_points)` — vectorised Bresenham ray-march; marks hits and frees in one batched write.
- `update_obstacles(clusters)` — matches incoming DBSCAN clusters to existing obstacles by centroid proximity (< 80 mm) and merges; stale obstacles (unseen for 5 frames) are removed.
- `vote_obstacle_color(obstacle, color)` — adds a vote for the given colour and subtracts one from the other; confirmation requires 7 net votes.
- `snapshot()` — returns deep copies of all three data structures under the lock. The lock is held only for the duration of the copy; all planning computation works on the copies.
- `update_lap_progress(pos)` — lap counter. The first position seen defines the start. A lap is counted each time the robot has left the start region (> 500 mm) and returned (< 300 mm). Returns the current lap count.

11.2 EgoInformation

File: `src/rpi/control/EgoInformation.py`

Maintains the robot's pose estimate $[x, y, \text{heading}]$ using an **Extended Kalman Filter** with two sensor sources.

State vector: $[x_{\text{mm}}, y_{\text{mm}}, \text{heading}_{\text{rad}}]$

Predict step — Ackermann odometry (called by ArduinoProcessor at 20 Hz):

Takes the encoder distance ds (mm) and current steering angle from the Arduino STATE message. If steering is near zero, propagates as straight-line motion. Otherwise uses the Ackermann kinematic model:

```
R_turn = wheelbase / tan(steering)
dh      = ds / R_turn
x_new   = x + R_turn * (sin(h + dh) - sin(h))
y_new   = y + R_turn * (-cos(h + dh) + cos(h))
h_new   = h + dh
```

The Jacobian F of this motion model is computed analytically and used to propagate the covariance matrix P :

```
P_new = F * P * FT + Q
```

where Q is a diagonal process noise matrix scaled by $|ds|$ using measured noise constants (`_PROCESS_NOISE_V`, `_PROCESS_NOISE_W`).

IMU correction step (called by ArduinoProcessor at 20 Hz):

The BNO055 absolute heading is treated as a scalar measurement. A scalar Kalman gain is computed from the heading covariance and measurement noise, and the entire state vector is updated along the heading direction. This corrects heading drift without requiring a separate lidar pose estimate on every cycle.

Lidar correction step (available for future use):

A full 3-DOF Kalman update for $[x, y, \text{heading}]$ simultaneously. Currently not wired up — the system relies on the IMU heading correction for real-time drift correction and replans on fresh map data.

Noise constant	Value	Unit
_PROCESS_NOISE_V	0.2005	mm per $\sqrt{\text{mm}}$
_PROCESS_NOISE_W	0.000321	rad per $\sqrt{\text{mm}}$
_MEAS_NOISE_IMU_HDG	0.001	rad
_MEAS_NOISE_LIDAR_XY	753.44	mm
_MEAS_NOISE_LIDAR_HD G	2.279	rad

12. Arduino Firmware

The Arduino Nano runs five cooperating modules. The main loop calls `navigator.update()`, `motor.update()`, and `Protocol::tick()` on every iteration (≥ 50 Hz). The IMU and encoder are read on demand within these calls.

12.1 Protocol

File: `src/arduino/Protocol.cpp/.h`

Implements the binary framing layer for UART communication with the Raspberry Pi.

Frame format:

```
[0xAA] [MSG_ID] [LEN] [PAYLOAD × LEN bytes] [CRC8]
```

CRC8 is computed over `MSG_ID + LEN + PAYLOAD` using polynomial `0x07`. On a framing error (bad start byte or CRC mismatch), bytes are discarded one at a time until the next `0xAA`.

Messages:

ID	Direction	Name	Payload
0x01	RPi → Arduino	SET_TARGET	forward_mm (f32), lateral_mm (f32), speed (f32) — 12 bytes
0x02	Arduino → RPi	STATE	heading (f32), speed (f32), steering (f32), distance (f32) — 16 bytes
0x03	RPi → Arduino	STOP	empty — 0 bytes

Watchdog: A 500 ms timer is reset on every valid `SET_TARGET` received. If the timer expires (RPI crashed or lost connection), the Arduino calls `navigator.stop()` unconditionally. This is the last-resort safety layer.

STATE reporting: Sent at 20 Hz (`STATE_REPORT_INTERVAL_MS = 20`). Carries BNO055 absolute heading, encoder-derived speed, current steering angle, and encoder-derived cumulative distance since the last `SET_TARGET`.

12.2 Navigator

File: `src/arduino/Navigator.cpp/.h`

The core motion controller. Maintains a body-frame target (`forward_mm`, `lateral_mm`) and drives the motor and steering to pursue it using a PID heading controller.

setTarget: Validates that the target is geometrically reachable given the robot's minimum turning radius. A target that falls inside either of the two circles of radius `wheelbase / tan(MAX_STEER_RAD)` centred on the robot's turn centres cannot be reached with Ackermann steering; such targets are rejected and the robot idles.

doPursuing (called each loop tick):

1. Read encoder ticks since last call and convert to mm. Accumulate body-frame position (`x`, `y`) using current IMU heading relative to the heading at `setTarget` time.
2. Compute distance to target. If within the arrival threshold (10 mm), brake and idle.
3. Compute heading error: `atan2(dy, dx) - current_heading`. When close to the target (within 100 mm), switch to a heading-hold mode to avoid oscillation.
4. Apply PID on the heading error ($K_p = -2.5$, $K_i = -0.7$, $K_d = -0.155$, filtered derivative with $\alpha = 0.7$, integral clamped to ± 0.43 rad).
5. Command the steering angle (clamped to ± 0.40 rad) and a speed profile that accelerates over the first 200 mm and decelerates over the last 500 mm.

Reverse: When `speed < 0`, the Navigator pursues in reverse. The steering sign is flipped (`REVERSE_STEER_SIGN = -1`) to compensate for the inverted geometry.

Parameter	Value
Wheelbase	89.625 mm
Heading Kp	−2.5
Heading Ki	−0.7
Heading Kd	−0.155
Integral clamp	±0.43 rad
Derivative filter α	0.7
Arrival threshold	10 mm
Hold radius	100 mm
Acceleration distance	200 mm
Deceleration distance	500 mm
Minimum speed	80 PWM units

12.3 Motor

File: `src/arduino/Motor.cpp/.h`

Controls the DC drive motor via two PWM pins (forward and reverse channels). Implements a **kickstart** mechanism: when the motor is asked to start from rest at a low speed, it briefly applies the minimum effective speed and then decays toward the requested speed at 200 units/s. This overcomes static friction without jerking.

`brake()` applies full PWM to both channels simultaneously, causing regenerative braking.
`stop()` turns both channels off for a coast stop.

Parameter	Value
Forward pin	D5
Reverse pin	D6
Motor direction	−1 (inverted wiring)
Kickstart decay	200 PWM units/s

12.4 Steering

File: `src/arduino/Steering.cpp/.h`

Controls the steering servo. Maps a requested angle in radians to a servo pulse width in microseconds, with asymmetric ranges for left and right because the servo's mechanical centre does not align with the electrical centre.

Parameter	Value
Servo pin	D9
Centre pulse	1050 μ s
Full-left pulse	1400 μ s
Full-right pulse	500 μ s
Maximum steering angle	± 0.40 rad ($\sim 23^\circ$)

12.5 Encoder

File: `src/arduino/Encoder.cpp/.h`

Counts quadrature encoder ticks on interrupt pins D2 and D3. Converts ticks to millimetres using `TICKS_PER_MM = 14.0`. Speed is computed by sampling the tick count every 50 ms and dividing by elapsed time. The `STATE` message reports `distance`: cumulative encoder distance since the last `SET_TARGET`, used by the RPi's Kalman filter predict step.

Parameter	Value
Encoder pins	D2, D3 (interrupts)
Ticks per mm	14.0
Speed sample interval	50 ms

12.6 IMU

File: `src/arduino/IMU.cpp/.h`

Interfaces with the BNO055 9-axis IMU over I²C. Uses the BNO055's on-chip sensor fusion mode, which outputs an absolute heading in degrees (0–360) fused from accelerometer, gyroscope, and magnetometer. The heading is converted to radians and wrapped to $[-\pi, \pi]$ before use. The Navigator reads the heading directly from `imuCtrl.getHeadingRad()` on each loop tick.

13. Calibration

All calibration scripts are in `calibration/`. They are run once on the assembled robot and their outputs are pasted into `main.py` or `EgoInformation.py`. Recalibration is needed if the camera, lidar, or motor assembly changes.

13.1 Camera Intrinsics

Script: `calibration/camera/camera-intrinsics.py`

Computes the camera's focal lengths and principal point using OpenCV's checkerboard calibration method.

Procedure:

1. Print a 9×6 inner-corner checkerboard with 24.5 mm squares.
2. Capture at least 15 photos of the checkerboard from different angles and distances using `calibration/camera/capture.py`.
3. Run the script:

```
python calibration/camera/camera-intrinsics.py <image_dir> <output_dir>
```

4. The script detects corners in each image (sub-pixel accuracy), discards images where detection fails, and calls `cv2.calibrateCamera`. A reprojection error below 0.5 pixels indicates good calibration.
5. Outputs `fx`, `fy`, `cx`, `cy`, `dist_coeffs` — paste into the `_CAMERA_INTRINSICS` constant in `main.py`.

Current values:

Parameter	Value
<code>fx</code>	658.35 px
<code>fy</code>	656.98 px
<code>cx</code>	312.90 px
<code>cy</code>	178.89 px

13.2 Camera Extrinsics

Manual measurement — no script required.

The extrinsics describe the camera's position and orientation relative to the robot's rear axle (the body-frame origin).

Parameter	Value	Description
x	0.0 mm	Lateral offset from centre line
y	134.70 mm	Forward distance from rear axle
z	31.07 mm	Height above ground
pitch	0.0°	Camera tilt angle
yaw	0.0°	Camera pan angle
roll	0.0°	Camera roll

Measure with a ruler on the assembled robot. Update `_CAMERA_EXTRINSICS` in `main.py` if the camera mount changes.

13.3 Obstacle Colour Ranges

Script: `calibration/camera/obstacle-colors.py`

Determines the HSV colour ranges for the red and green competition pillars under the specific lighting conditions of the competition venue. Lighting varies significantly between venues, so this calibration should be repeated before each competition.

Procedure:

1. Place the robot in front of both a red and a green pillar under competition lighting.
2. Capture a photo with the camera.
3. Run the script with the photo:

```
python calibration/camera/obstacle-colors.py <image>
```

4. Draw a rectangle over the red pillar in the GUI, then over the green pillar.
5. The script computes HSV min/max ranges with a configurable tolerance (default ± 10). For red, it handles the hue wrap-around at $0^\circ/180^\circ$ by splitting into two ranges.
6. Copy the printed `red_lower_1`, `red_upper_1`, etc. into `_OBSTACLE_COLOR_RANGES` in `main.py`.

13.4 Odometry Noise

Script: `calibration/odometry_noise.py`

Measures the Kalman filter's process noise constants `_PROCESS_NOISE_V` (position noise) and `_PROCESS_NOISE_W` (heading noise) by running the robot straight and comparing encoder-reported distance against physically measured distance.

Procedure:

1. Mark a start position and a target 1000 mm ahead on the competition surface.
2. Run the script:

```
cd src/rpi && python ../../calibration/odometry_noise.py --runs 10 --distance 1000
```

3. After each run the script prompts for the measured forward distance and lateral offset.
4. It computes `_PROCESS_NOISE_V = std(position_error) /  $\sqrt{\text{distance}}$` and `_PROCESS_NOISE_W = std(heading_error) /  $\sqrt{\text{distance}}$` .
5. Update these values in `EgoInformation.py`.

13.5 IMU Noise

Script: `calibration/imu_noise.py`

Measures `_MEAS_NOISE_IMU_HDG` by holding the robot stationary and logging BNO055 heading samples to compute the standard deviation.

Procedure: Run the script while the robot is stationary for approximately 60 seconds. Copy the reported heading standard deviation (in radians) into `EgoInformation._MEAS_NOISE_IMU_HDG`.

13.6 LiDAR Noise

Script: `calibration/lidar_noise.py`

Measures range noise by scanning a flat wall at a known distance across multiple scans and computing the standard deviation of returns. The result can be used to tune `_MEAS_NOISE_LIDAR_XY` if a lidar-based position correction step is added in a future version.